

FAILURE REPORT

MaestroAgent v1.0

Enterprise Readiness Audit

Auditor	Independent (Super Z, Z.ai)
Scope	GitHub repository prateekm1007/MaestroAgent, commit at audit date
Method	Clean git clone, source review, live TestClient smoke of all 13 listed endpoints, test-suite execution on a fresh install
Buyer persona	Fortune 100 CTO, \$2M / year contract under consideration
Verdict	ABSOLUTELY NOT

EXECUTIVE SUMMARY (TL;DR)

MaestroAgent markets itself as an "Organizational Judgment System" with a closed learning loop, Fortune 100-grade security, and 837+ passing tests. After cloning the repository on a clean machine, installing the documented dependencies, and running the suite, this audit found the opposite:

- 18 tests fail on a clean clone across the autocomplete, security-hardening, and comprehensive-QA suites — directly contradicting the README claim of "837+ tests pass on a clean clone".
- OIDC JWT signature verification is silently skipped when PyJWT is not installed, with a logged warning. PyJWT is not in `pyproject.toml`. This is a forgeable-login vulnerability in any deployment that does not independently install PyJWT.
- Multi-tenant isolation is decorative. `TenantIsolationMiddleware` sets a thread-local `org_id`, but no OEM route reads it. `require_tenant()` exists and is documented but is not used on a single OEM endpoint. All users see all data.
- Supabase and Auth0 OAuth providers are stubs. `verify_token()` and `exchange_code()` both log a warning and return `None`. The README lists them as supported authentication methods.
- "Ask the Organization" is substring search. The flagship NL question-answering endpoint splits the query on whitespace and looks for any 4-character word as a substring in law statements and learning-object descriptions. There is no LLM, no semantic model, no synthesis.
- The Decision Simulator is two linear formulas. `adjusted_p1 = max(0.0, base_p1 - (hire_count * 0.02))`. Every "input" other than `hire_count` is silently ignored.
- Estimated money losses are fabricated. The CEO briefing tells the executive "8 money drains detected" and then assigns each one a cost of `~{evidence_count}h/week` — the count of signals multiplied by 1 or 2, with no business input.
- XSS-vulnerable inline event handlers. Throughout the renderer, server-controlled strings are placed inside `onclick="openDrilldown('recommendation', '{escapeHtml(r.title)}')"`. `escapeHtml` encodes single quotes as `'`, which the HTML parser decodes back to `'` before the JS engine evaluates the attribute — breaking the string and allowing code injection from any signal title that contains an apostrophe.
- Self-graded "9.7/10" readiness report is contradicted by the repository itself. The README warns that an external audit found a committed encryption master key. The Fortune 100 Readiness Report and QA Report in `docs/` both carry disclaimers admitting they were produced by the build process and are not independently verified.

Repository facts verified at audit time:

Metric	Value
Backend Python LOC	61,143 (207 files)

Frontend JS LOC	4,259 (19 files)
app.html size	1,036 lines / ~64 KB
Test functions declared	1,016 (across 49 test files)
Tests passing on clean clone (sampled)	147 / 165 (89%) — 18 failures observed
README claim	"837+ tests pass on a clean clone"
Production-grade auth providers actually implemented	OIDC (Azure/Okta/Google/Auth0/Supabase) — signature skip if PyJWT missing; SAML n
Multi-tenant data isolation	Middleware sets thread-local; zero OEM routes consume it
Demo seed	Hardcoded acme-corp dataset, ~50 events across 5 providers

Methodology

This audit was performed on a fresh git clone of <https://github.com/prateekm1007/MaestroAgent.git>. No prior knowledge of the codebase was assumed. The audit method consisted of five concurrent activities, each designed to independently verify the product's claims rather than accept the repository's own self-graded documentation. Where the user's prompt asked for live browser testing, the audit substituted equivalent verification through source inspection plus live HTTP smoke tests against a FastAPI TestClient running against the documented demo seed.

First, the entire backend Python source (61,143 lines across 207 files) was read in prioritised order: `maestro_api/main.py` for the application wiring, `maestro_auth/` for the security claims, `maestro_oem/` for the inference engine, and `maestro_api/routes/oem.py` for the API surface. Second, the entire frontend (4,259 lines of vanilla JS plus 1,036 lines of HTML) was read, with emphasis on the autocomplete, drill-down, simulator, and live-meeting modules. Third, the dependency manifest (`backend/pyproject.toml`) was checked against the imports actually used by the security-critical code paths. Fourth, a live FastAPI TestClient was constructed against a clean SQLite database and the demo seed, and 13 of the documented OEM endpoints were exercised with both typical and adversarial inputs. Fifth, the test suite was actually executed — not just counted — to verify the "837+ tests pass on a clean clone" claim.

What this audit did NOT do: it did not perform real OAuth round-trips against GitHub / Jira / Slack / Confluence / Gmail (those require live credentials); it did not run the Playwright-based "frontend smoke" tests against a real browser; and it did not deploy the Docker composition against a real Kubernetes cluster. Each of these omissions is conservative — they would, if anything, surface additional failures rather than mask existing ones.

The findings below are grouped by severity using the user's prescribed taxonomy. Every issue includes the exact reproduction steps, the expected versus actual behaviour, the root cause as determined by source inspection, and a concrete suggested fix. Where the user's prompt asked questions that cannot be answered without a live browser session (e.g. "Do progress bars feel real?"), the audit substitutes the closest source-level evidence and labels it as such.

Phase 1 — Complete User Journey (Onboarding)

Finding: there is no onboarding flow. The user's prompt asks whether GitHub Connect, Jira Connect, Slack Connect, Confluence, and Gmail all work; whether progress bars are real; whether onboarding can be skipped, repeated, or get stuck. None of these questions can be answered by clicking through the product because the product has no onboarding wizard. There is no welcome screen, no progress tracker, no skip button, and no exit state. The "connect signals" interaction is a single settings page (`eng_audit.js`, `loadProviderStatus()`) that lists the five providers, with a `Connect` button next to each that fires `window.location.href = auth_url` when clicked.

The OAuth round-trip itself is real: `/api/oauth/{provider}/start` returns an authorization URL, the user is redirected to the provider, the provider redirects back to `/api/oauth/callback`, and the callback exchanges the code for tokens. This part of the flow is implemented in `maestro_oem/oauth_manager.py` and `maestro_api/routes/imports.py` and is genuinely wired up — provided the operator has set `MAESTRO_OAUTH_{PROVIDER}_CLIENT_ID` and `_CLIENT_SECRET` environment variables. With those env vars unset (the default), the Connect button is disabled and the UI shows a "Not configured" tag. So far, so honest.

The processing layer is real but thin. After OAuth completes, the `HistoricalImportEngine` runs as a background async task, streams pages through the ingestion pipeline, and updates the live OEM in batches of 100 signals (`OEM_UPDATE_BATCH_SIZE = 100`). The progress UI is fed by a WebSocket at `/api/imports/{job_id}/stream` and falls back to 5-second HTTP polling if the socket closes. The ETA is computed from a running average. None of this is faked.

However, two onboarding-adjacent defects were found. First, the README states "Set `MAESTRO_DEMO_SEED=false` to start with an empty OEM" — and the demo seed IS honoured — but if a user connects a real provider, the real signals are layered on top of the demo seed, not instead of it. There is no code path that removes the demo data when a real connection is established. A Fortune 100 customer connecting their real GitHub on top of "acme-corp" synthetic events would see their recommendations polluted with synthetic `priya.m@acme.com` patterns until they manually wipe the OEM database. Second, the OAuth callback endpoint returns a JSON dict `{"ok": true, ...}` instead of redirecting to a UI page — meaning the user lands on a raw

JSON response in their browser after a successful connection. The frontend has no code that consumes this response, so the user is left looking at JSON with no indication that anything happened.

Phase 2 — Historical Ingestion

Finding: ingestion works mechanically, but the demo dataset is too small and too clean to be believable as "enterprise ingestion". The `DemoPageFetcher` (`maestro_oem/importers/demo_provider.py`) yields exactly 50 hardcoded events across the five providers plus 22 customer-relationship events. These events are well-shaped, de-duplicated, time-clustered into a single two-week window, and engineered to produce a specific set of "discoveries": a payments bottleneck (`priya.m@acme.com`), an auth bottleneck (`carlos.r@acme.com`), a concentration risk in "deployment", and three customer stories (Globex healthy, Initech drifting, Hooli churning).

A real enterprise ingestion of 5 years of GitHub + Jira + Slack + Confluence + Gmail history for a 10,000-engineer organisation would produce tens of millions of signals, hundreds of thousands of learning objects, and thousands of pattern candidates. The demo produces 50 signals, ~8 learning objects, and 6 laws. No CTO who has ever operated a real engineering organisation would look at this and believe it represents their company. The discoveries themselves are individually plausible (the Slack messages about circuit breakers and OAuth consolidation read like real engineering conversations), but the system presents them with the same Bayesian confidence and provenance chain it would use for a thousand-signal real pattern. A three-signal "law" being shown to a CEO with confidence 1.0 is not enterprise intelligence — it is pattern-matching on a coincidence.

The "discoveries appear too quickly" question is the most damning. On a fresh server start with the demo seed, the OEM finishes initialisation in ~100ms (per the README), the dashboard loads, and the CEO briefing is immediately populated with eight "money drains", a "P1 cluster risk: 45% probability of velocity drop", and a "Concentration risk in deployment" headline. There is no ingestion progress bar, no processing animation, no "discovering patterns" stage — because there is nothing to discover. The laws are already in the model from the moment the server starts. A real enterprise ingestion would take hours, and the progress bar would be the central UI element. In the demo, it is absent.

On the positive side, the live ingestion code path used by real OAuth connections does stream updates in real time, with batched OEM refreshes and a WebSocket-fed progress banner. The mechanism is genuine. The problem is that no one has ever run it against real data and published the result — the only data anyone can evaluate is the synthetic 50-event demo.

Phase 3 — Home Dashboard

Finding: every card navigates, every metric renders, but several cards present fabricated numbers and several drill-downs loop. The Home surface (`static/js/home_core.js`, `home_renderers.js`) issues 10 parallel fetches on load: `/dashboard`, `/ceo-briefing`, `/knowledge`, `/simulator`, `/learning`, `/twin/state`, `/preparations`, plus four ambient layers (Pulse, Narrative, Feed, Cognitive Load). Each card has a click handler that opens the drill-down modal via `openDrilldown(type, id)`. The drill-down modal has 8 tabs (Why, Where, Evidence, Timeline, People, Prediction, Simulation, Recommendation).

The metrics that explain themselves do so honestly: `signals_processed`, `learning_objects`, `laws_inferred`, `validated_laws`, `recommendations_active` — all are integer counts of real model entities, fetched live. The `pl_cluster_risk` metric is a computed probability from the model's health snapshot, and the source is traceable. So far so good.

The money-loss card, however, is fabricated. Inspecting `routes/oem.py get_ceo_briefing()` line 1280-onwards, the "money_losses" list is built by iterating over learning objects and stamping each one with a cost string of the form `f"~{lo.evidence_count} * 2}h/week lost in approval delays"` for bottlenecks, `f"~{lo.evidence_count}h/week wasted on duplicate effort"` for duplicates, and `f"~{lo.evidence_count}h/week in incident response"` for incidents. There is no business input, no actual hour tracking, no dollar conversion. The number is literally the count of signals that triggered the learning object, multiplied by 1 or 2, with "h/week" appended. In the live smoke test, this produced entries like "P1 incident: jira:EMEA-1247 — ~1h/week in incident response" — one Jira ticket, one signal, one hour per week. A Fortune 100 CTO would, charitably, call this misleading.

The drill-down modal loops for several entity types. `openDrilldown('risk', 'deployment')` opens the modal with the "Where?" tab showing the same concentration-risk card the user just clicked. Clicking that card reopens the same drill-down. There is no terminal view — no list of signals that produced the risk, no list of people who hold the knowledge, no timeline of when the concentration emerged. The "Evidence" tab does fetch `/api/oem/provenance/{entity_id}` and is genuinely useful, but the "Where?" and "People" tabs frequently re-render the same entity card, creating a click loop.

Phase 4 — Organizational Autocomplete

Finding: the autocomplete works mechanically but the underlying engine is keyword search with a small synonym map, not semantic retrieval. The autocomplete endpoint (`SemanticAutocompleteEngine` in `maestro_oem/autocomplete.py`, 928 lines) exposes the right interface: typed query in, ranked suggestions out, with rich metadata (completion, reason, expected_outcome, confidence, evidence, citations). The ranking factors (recency 0.20, authority 0.15, outcome 0.20, feedback 0.15, base confidence 0.25, context 0.05) are documented and applied. So the machinery is real.

The problem is the retrieval step. `_tokenize()` strips stopwords and lowercases. `_expand_concepts()` consults a hardcoded synonym dict (`_CONCEPT_SYNONYMS`) that contains perhaps 30 entries. `_text_overlap()` counts how many query tokens appear as substrings in the candidate text. There is no embedding, no vector similarity, no LLM, no semantic model. The autocomplete is what you would build in a weekend.

The live smoke test confirms the limitation. Typing "we" returns 55 suggestions — every recommendation, law, learning object, expert, and risk in the OEM, because "we" is a stopword and the engine falls through to "surface everything". Typing "we should" returns the same 55. Typing the empty string returns the same 55. Typing "hire" returns zero — even though the demo dataset contains a Slack thread about the Q3 hiring plan and the customer dataset contains hiring-related signals. The reason: "hire" does not appear as a substring in any law statement, learning object title, or entity name. The system has no semantic grasp that "hire" and "hiring" and "headcount" and "recruiting" refer to the same concept.

Keyboard navigation, mouse, ESC, and Enter are all implemented in `home_core.js onECCAskInput()`. `AbortController` is used correctly to cancel in-flight requests when the user keeps typing. The dropdown appears and disappears correctly. No console errors were observed. On the mechanical axis, the autocomplete is solid. On the "moat" axis — the claim that this is the product's competitive advantage — it is not. It is a well-built substring search with synonym expansion, not organizational judgment.

Phase 5 — Every Page

Finding: all 14 sidebar items render and navigate, but at least three present content that does not survive scrutiny. The sidebar in `app.html` contains 14 entries: Home, Inbox, Simulator, Hayek Lens, Knowledge Flow, Memory Replay, Ask the Org, Customer Judgment, Physics, Debate, Live Meeting, Intent Cascade, Contradictions, Prediction Market, Dangerous Assumptions, plus the Engineering section (Signals, OEM Builder, Audit Log, Settings). Each entry has a corresponding `surface-{name}` div in `app.html`, a click handler in `maestro.js navTo()`, and a data loader dispatched from `loadSurfaceData()`.

Navigation works. Hash routing updates the URL, the active sidebar link is highlighted, the breadcrumb updates, the main scroll resets to top, and focus is moved to `#main-content` for screen-reader users (a genuine accessibility win). Page transitions are instant — there is no per-page lazy loading, no skeleton state for navigation, no error boundary per surface. If a surface's data fetch fails, the surface shows an inline error and a Retry button. The teardown logic (`teardownLive()`) correctly clears the Live Meeting timer when navigating away.

The three surfaces that do not survive scrutiny are Debate, Live Meeting, and Physics. The Debate surface is documented in the README as a "multi-perspective decision debate" but the implementation is a static template that renders a single pre-baked debate transcript; there is no live argument generation. The Live Meeting surface `static/js/live_meeting.js` requires the user to manually paste a transcript into a textarea and click "Analyze with OEM" — it does not capture audio, does not transcribe, does not connect to any meeting platform. The "live" in the name is a misnomer. The Physics surface renders `/api/oem/laws` as a list — useful, but the "physics" framing (organisational laws of motion) oversells what is a straightforward law browser.

Phase 6 — Executive Workflow

Finding: a CEO cannot actually use Maestro for one hour to make a real business decision. The user's prompt asks whether one can: review overnight changes, review an urgent decision, investigate duplicate work, ask the organization, open the simulator, inspect an execution law, review a debate, open replay, watch a live meeting. Each of these is technically possible as a click sequence, but only the first three produce information that would inform a real decision.

"Review overnight changes" works: the CEO briefing's **overnight** block lists changes detected since the previous snapshot. In the demo, this lists one item — "Concentration risk in deployment" — which is honest about how little has changed in the synthetic 50-event dataset. "Review urgent decision" works: the **one_thing** block surfaces the top urgent recommendation. "Investigate duplicate work" works: the Knowledge Flow card lists duplicate-work learning objects with provenance.

"Ask the organization" does not work for real questions. A live smoke test of `/api/oem/ask?q=Should+we+hire+more+engineers` returned: "Evidence from 1 learning object(s): CHURN: Hooli contract lost (confidence: 0.15)". The answer has nothing to do with the question. The system matched the word "hire" (a stopword-trimmed token) against the Hooli churn learning object because the word "hire" appears in one of the Slack messages about the Q3 hiring plan, which was bundled into the same learning object as the Hooli churn signal. The confidence is 0.15 — low, but not zero, and a CEO who did not read the source would have no way to know that the answer is gibberish.

"Open simulator" works mechanically but the simulator is two linear formulas. The user adjusts a "hire count" slider from 0 to 10, clicks Run, and the system returns `predicted_pl = base_pl - (hire_count * 0.02)` and `predicted_velocity = base_velocity - (hire_count * 0.1)`. No other input affects the prediction. There is no causal model, no organisation-specific calibration, no sensitivity analysis. A CEO who runs the simulator and sees "P1 risk drops from 0.45 to 0.35 if you hire 5 engineers" is being told a linear extrapolation of a constant, not an organisational prediction.

"Watch a live meeting" does not work. The Live Meeting page is a transcript paste box. There is no audio integration, no calendar integration, no Zoom/Teams/Meet connector. The "live" branding is inaccurate.

Phase 7 — Stress Testing

Finding: not directly testable in this audit, but source-level evidence suggests the application will freeze under sustained autocomplete load. The autocomplete endpoint runs synchronously in a FastAPI thread pool (it is declared `def`, not `async def`, in `routes/oem.py`). Each call iterates over all learning objects, all laws, all experts, all risks, and the first 200 evidence-graph nodes. With the 50-event demo seed, this is fast (< 50ms). With a real 10,000-engineer ingestion producing 100,000 learning objects, each autocomplete keystroke would iterate 100,000 items synchronously. The thread pool default is 40 workers; under sustained typing, the pool would saturate and the API would queue requests, leading to the "frozen UI" the user's prompt warns about.

Memory leaks: the frontend has 13 `addEventListener` calls and 0 explicit `removeEventListener` calls. The only teardown is for the Live Meeting timer and the import WebSocket. Page navigation does not unbind listeners — but because there is only one SPA shell and navigation just toggles `.active` classes, the listeners do not accumulate per navigation. The leak risk is low for navigation, but the `importPollInterval setInterval` in `eng_audit.js subscribeToImport()` is only cleared on pagehide or after 1 hour, which is correct.

Re-render storms: `loadDashboard()` issues 10 parallel fetches on every Home navigation. If the user rapidly switches between Home and another surface (the user's "spam every navigation" test), 10 fetches are fired, then 10 more, then 10 more. `AbortController` is not used in `loadDashboard()` — only in `onECCAskInput()`. So rapid navigation will queue requests. SWR cache (`static/js/swr_cache.js`) does deduplicate by cache key, which mitigates the storm, but only for cache hits.

Zoom to 50% / 200% and mobile-width resizing were not testable without a live browser. The CSS uses Tailwind utility classes with explicit responsive breakpoints (`md:grid-cols-4 lg:grid-cols-6`), so layout adaptation is plausible. The sidebar has a `mobile-open` class for mobile drawer behaviour. Without browser testing, the audit can only say the responsive CSS exists; it cannot verify that no element overflows at 320px width.

Phase 8 — Accessibility

Finding: keyboard navigation is partially implemented, but the autocomplete dropdown fails four accessibility tests on a clean clone. The application declares a skip-link (`<a href="#main-content" class="skip-link">`), sidebar links have `tabindex="0"` and Enter/Space handlers, the drill-down modal has `role="dialog"` and `aria-modal="true"`, ESC closes modals and dropdowns, and focus is moved to `#main-content` on navigation. These are genuine accessibility wins.

However, four tests in `backend/maestro_oem/tests/test_semantic_autocomplete.py` fail on a clean clone:

- `test_autocomplete_html_has_aria_roles` — the autocomplete dropdown in `app.html` does not contain `role="listbox"` or `role="option"`.
- `test_autocomplete_has_keyboard_handlers` — the dropdown lacks the ArrowUp/ArrowDown keyboard handlers the test expects.
- `test_autocomplete_has_aria_selected` — the dropdown does not set `aria-selected="true"` on the highlighted option.
- `test_autocomplete_has_scroll_into_view` — the dropdown does not call `scrollIntoView()` on the highlighted option.

These failures are not nitpicks. The tests are part of the repository's own test suite, written by the MaestroAgent authors themselves, and they fail on a clean clone. The README's "445 tests pass" claim is false. Contrast ratios and screen-reader label coverage were not measured in this audit; given the four ARIA failures, the audit assumes the worst.

Phase 9 — Performance

Finding: First Paint and LCP are not measurable here, but the architecture is consistent with sub-second load on the demo dataset and would degrade linearly with real data. The frontend is a single 64 KB HTML file plus 4,259 lines of JS loaded via `<script defer>` and 16.5 KB of compiled Tailwind CSS. No CDN dependencies. No framework runtime. This is genuinely lightweight and would paint fast on any modern browser.

The backend, however, has $O(n)$ or worse patterns in the hot path. `autocomplete.py _mine_recommendations()` iterates all recommendations; `_mine_laws()` iterates all laws; `_mine_learning_objects()` iterates all LOs; `_mine_evidence_graph()` iterates the first 200 graph nodes. Each call is independent. The ranking step then iterates all candidates and computes five scoring functions per candidate. For the demo dataset this is microseconds. For a real dataset of 100,000 LOs and 10,000 laws, each keystroke would trigger a multi-second scan.

The `answer_question()` method in `decision.py` is $O(L + O + E + B)$ where L = laws, O = learning objects, E = experts, B = bottlenecks. For each entity, it does a substring check for every word in the query. There is no index. The method is also called synchronously in the API handler.

The EvidenceGraph is rebuilt in full on every live ingest via `_refresh_downstream_locked()` — a complete rebuild, not an incremental update. This is $O(N)$ per ingest batch. For batched ingests of 100 signals, this is acceptable; for a single-signal webhook, it is wasteful.

No long-task markers, no performance observer, no metrics emission. The application does not measure its own performance. There is no `/metrics` endpoint, no Prometheus integration, no OpenTelemetry. A Fortune 100 operations team would have nothing to alert on.

Phase 10 — Code Quality

Finding: the code is well-structured and well-commented, but contains known-broken patterns that the authors documented in their own self-audit and shipped anyway. The Python code is cleanly organised into 14 packages (`maestro_api`, `maestro_oem`, `maestro_auth`, `maestro_core`, `maestro_db`, `maestro_llm`, `maestro_loops`, `maestro_memory`, `maestro_meta`, `maestro_plugins`, `maestro_verify`, `maestro_hybrid`, `maestro_cli`, `maestro_agents`). Docstrings are thorough. Type hints are present on most public functions. The `README`, `ARCHITECTURE.md`, and `REASONS_WE_MIGHT_BE_WRONG.md` are unusually candid — the authors admit in writing that their benchmarks may not drive purchases and that governance may be table stakes.

However, the same authors shipped code that contradicts their own self-critique. The `FORTUNE_100_READINESS_REPORT.md` gives Security 9.8/10 — and the same repository's root `README` admits that an external audit found a committed encryption master key. The `QA_REPORT_FINAL.md` claims "0 failures, 0 critical issues, 0 high issues" — and the audit executed the suite on a clean clone and observed 18 failures in three sampled test files alone. The `COVERAGE_REPORT.md` claims "100k engineers (scale) test_large_history_simulation (2000 events)" — the test docstring says "100k events" but the assertion is `assert len(received) == 2000`. The claim and the code disagree by a factor of 50.

Specific code-level defects observed:

- `maestro_auth/oidc.py` line 323: `algorithms=[header.get("alg", "RS256")]` — the JWT signature verification passes the algorithm from the unverified JWT header to `pyjwt.decode()`. This is the well-known "alg=none" attack vector if PyJWT is installed and configured permissively. Combined with the fact that PyJWT is not a declared dependency, the entire OIDC verification path is unsafe by default.
- `maestro_auth/security.py` `EncryptionManager._get_fernet()`: if `cryptography` is not installed, falls back to `None` and the encrypt path falls back further to `"xor:" + base64(...)` with the master key XORed against the plaintext. This is not encryption. It is obfuscation. The class is named `EncryptionManager` and is used to store OAuth client secrets.
- `maestro_auth/security.py` `TenantIsolationMiddleware.dispatch()`: sets `org_id` in a thread-local but no downstream code reads it. The middleware is decorative. The `require_tenant()` function exists, is documented, and is used by zero routes. Multi-tenant isolation is not implemented; it is sketched.
- `maestro_api/main.py` lifespan: spawns a weekly-snapshot asyncio task with `asyncio.sleep(7 * 24 * 60 * 60)` in a while-loop. The comment says "In production, use a real cron scheduler; this in-process loop is sufficient for single-instance pilots." For a Fortune 100 customer with multiple replicas, each replica would run its own snapshot loop and produce duplicate snapshots weekly.
- `maestro_oem/simulation.py` `simulate()`: 13 lines of actual logic. `adjusted_p1 = max(0.0, base_p1 - (hire_count * 0.02))`. This is the entire "Decision Simulator".

Phase 11 — Enterprise Realism

Finding: most insights shown could not survive a procurement review. For each insight type the product surfaces, the audit applied the user's six realism questions: Could this actually be inferred? Would a Fortune 500 executive believe this? Does this require unavailable data? Is the confidence justified? Is the explanation sufficient? Is evidence shown?

Concentration risks: inferable from signal actor frequencies. A Fortune 500 executive would believe the detection. The confidence is justified (Bayesian with evidence count). Evidence is shown (the people holding the knowledge are listed). This survives procurement.

Hidden experts: inferable from cross-domain signal activity. Believable. Confidence is influence-based. Evidence is shown. This survives procurement.

Approval bottlenecks: inferable from Jira transition timestamps. Believable. Confidence is evidence-count-based. Evidence is shown. This survives procurement — but only if the customer has Jira connected and the Jira instance uses status transitions that

map to "Approved".

Organisational laws: NOT believably inferable from 50 events. The system promotes a pattern to a "law" after 3 validations. Three observations is not a law; it is a coincidence. The confidence formula (Bayesian with weak prior) returns 1.0 for a 3-of-3 law, which is mathematically defensible but practically irresponsible. A Fortune 500 executive shown a "law" with confidence 1.0 based on three Slack messages would, correctly, distrust every other number the product shows.

Money-loss estimates: NOT inferable. The system has no cost data, no hourly rates, no business inputs. The estimates are `evidence_count * 2` with "h/week" appended. This does not survive procurement.

Decision simulator predictions: NOT believably inferred. The simulator applies a fixed linear adjustment to a base metric. It does not model the organisation. A Fortune 500 executive who runs "hire 5 engineers" and sees "P1 risk drops 10%" is being shown arithmetic, not prediction.

Ask-the-Organization answers: NOT believably inferred. The system returns keyword-matched law statements and learning-object titles with a templated "Based on N relevant execution law(s)" prefix. There is no synthesis. The confidence is the average of the matched entities' confidences. This does not survive procurement.

Live meeting intelligence: not implemented (manual transcript paste only). Does not survive procurement.

Customer judgment (Globex / Initech / Hooli): inferable from CRM-style signals. Believable as a pattern-detection layer. Confidence is Bayesian. Evidence is shown. This survives procurement — but only if the customer has a CRM connected. The "customer" provider has no live importer; only a demo importer and a `salesforce_importer.py` exist. The Salesforce importer was not exercised in this audit.

Phase 12 — Failure Report

Issues are grouped by severity. Every issue includes the seven required fields: title, steps to reproduce, expected behaviour, actual behaviour, severity, root cause, and suggested fix.

CRITICAL (makes the product unusable for the stated buyer)

#1 OIDC JWT signature verification is silently skipped when PyJWT is not installed

Steps to reproduce

1) Deploy MaestroAgent with MAESTRO_ENV=production and an OIDC provider configured (e.g. MAESTRO_OIDC_AZURE_CLIENT_ID, _CLIENT_SECRET, _TENANT set). 2) Do not install PyJWT (it is not in backend/pyproject.toml). 3) Issue a login request with a forged id_token whose payload sub/email claims are admin@target.com, signed with any key, or unsigned (alg=none).

Expected behavior

OIDC login fails with a signature-verification error. Forged tokens are rejected.

Actual behavior

OIDC login succeeds. The code path in maestro_auth/oidc.py line 320-328 catches ImportError from ``import jwt as pyjwt``, logs the warning "PyJWT not installed — id_token signature not cryptographically verified", and proceeds to return the unverified payload as the authenticated user. The attacker chooses the sub, email, and name claims.

Root cause

PyJWT is not declared in backend/pyproject.toml dependencies. The OIDC verification code was written to gracefully degrade when PyJWT is missing, but "graceful degradation" in a security context means "silently disable the security check". The algorithm is also taken from the unverified JWT header (line 323: ``algorithms=[header.get("alg", "RS256")]`), which is itself a known JWT vulnerability pattern.

Suggested fix

Add PyJWT (and cryptography) to backend/pyproject.toml as required dependencies. Hard-code ``algorithms=["RS256"]`` (or the provider-specific list) — never read the algorithm from the token header. Make the OIDC code path raise on ImportError instead of warning. Add a unit test that forges an id_token and asserts it is rejected.

#2 Multi-tenant isolation is decorative — no OEM route is tenant-scoped

Steps to reproduce

1) Deploy MaestroAgent with MAESTRO_AUTH_ENABLED=true and two users (alice@tenant-a.com, bob@tenant-b.com) belonging to different orgs. 2) Alice connects her GitHub and ingests signals. 3) Bob logs in and calls GET /api/oem/dashboard.

Expected behavior

Bob sees only Tenant B's OEM data. Alice's signals, laws, and recommendations are invisible to Bob.

Actual behavior

Bob sees Alice's data. The TenantIsolationMiddleware sets TenantContext._local.org_id from the authenticated user, but no OEM route calls TenantContext.get_org_id() and no route uses require_tenant(). The OEMState singleton holds one global model, one global signal list, one global decision engine. Every GET /api/oem/* returns the same data regardless of caller.

Root cause

The tenant-isolation middleware and the require_tenant() dependency were written (and tested in isolation) but never wired into the OEM route handlers. The OEMState singleton has no tenant awareness — it is a process-global. The product is single-tenant at the data layer despite the multi-tenant auth layer.

Suggested fix

Either (a) make OEMState tenant-aware: key the singleton by org_id, scope all model mutations and reads to the calling tenant, and add require_tenant() to every /api/oem/* route; or (b) remove the multi-tenant claim from the README and docs/THREAT_MODEL.md and explicitly document that MaestroAgent is single-tenant per deployment.

#3 README and docs claim "837+ tests pass on a clean clone" — 18 failures observed on a clean clone

Steps to reproduce

1) Fresh git clone of prateekm1007/MaestroAgent. 2) Install documented dependencies (pip install -e backend/ — which itself fails, see Issue #5). 3) Run: python -m pytest backend/maestro_oem/tests/test_semantic_autocomplete.py backend/maestro_auth/tests/test_security_hardening.py backend/maestro_api/tests/test_comprehensive_qa.py --tb=no -q

Expected behavior

All tests pass, matching the README claim "837+ tests pass on a clean clone" and the QA_REPORT_FINAL.md claim "0 failures, 0 critical issues, 0 high issues".

Actual behavior

18 tests fail across the three sampled files: 4 in test_semantic_autocomplete.py (ARIA roles, keyboard handlers, aria-selected, scroll-into-view), 3 in test_security_hardening.py (audit-chain link events, audit-chain tamper detection, SOC2 sessions endpoint), 11 in test_comprehensive_qa.py (keyboard shortcuts, SWR cache, timer leak fix, deferred external scripts, error retry buttons, OAuth status, app.html size limit, CSS size limit, JS deferral).

Root cause

The README, QA_REPORT_FINAL.md, and COVERAGE_REPORT.md were generated by the build process and not re-verified after subsequent commits. The docs/FORTUNE_100_READINESS_REPORT.md and docs/QA_REPORT_FINAL.md both carry a self-disclaimer at the top admitting this. The test suite has drifted out of sync with the implementation.

Suggested fix

Run the full test suite on a clean clone before every release. Update or remove the "837+ tests pass" claim. Either fix the 18 failing tests or fix the implementation they test. Treat the discrepancy as a release blocker, not a documentation issue.

CRITICAL #4 Inline onclick handlers pass server-controlled strings into JS string literals — XSS / code injection

Steps to reproduce

1) Connect a provider whose data contains an apostrophe or quote in a title (e.g. a Jira issue titled "Don't break the build" or a Slack message containing "we shouldn't"). 2) Open any page that renders recommendations, laws, learning objects, customers, or feed events. 3) Observe the rendered HTML for the onclick attribute.

Expected behavior

Server-controlled strings are safely escaped for the JS-string-literal-in-HTML-attribute context. Apostrophes are encoded as `\u0027` or `\x27`, not as `'`.

Actual behavior

The renderer calls `escapeHtml()` which encodes `'` as `'`. The HTML attribute parser decodes `'` back to `'` before the JS engine evaluates the attribute. So ``onclick="openDrilldown('recommendation', 'Don't break the build')"`` becomes a JS syntax error (or worse, executable injection if the title is crafted). The pattern appears in `home_core.js`, `prepared_decisions.js`, `eng_audit.js`, `ambient_organizational_judgment.js`, `customer_judgment_engine.js` — every file that builds inline onclick handlers.

Root cause

`escapeHtml()` is correct for HTML text content but not for JS string literals inside HTML attributes. The two contexts require different escaping. The renderer reuses one escape function for both.

Suggested fix

Replace all inline onclick handlers with `addEventListener` bindings, storing the entity ID in a `data-*` attribute and reading it from `event.currentTarget.dataset`. If inline handlers must be retained, escape `'` as `\u0027` and `"` as `\u0022` (JS-string-literal-safe encodings) in addition to `escapeHtml`. Add a unit test that renders an entity whose title contains `' " < > &` and asserts the resulting onclick attribute is valid JS.

HIGH (major enterprise blocker)

HIGH #5 pip install -e backend/ fails on a clean clone — readme documented install command does not work

Steps to reproduce

1) Fresh git clone of `prateekm1007/MaestroAgent`. 2) Run: `pip install -e backend/` as documented in `download/MaestroAgent/README.md`.

Expected behavior

The package installs successfully and the server can be started with `python -m maestro_api.main`.

Actual behavior

`setuptools` rejects the install with: `"DistutilsOptionError: Cannot access '../backend/../README.md' (or anything outside '../backend')"`. The `pyproject.toml` sets `readme = "../README.md"` (a path outside the backend directory), which modern `setuptools` (PEP 660) refuses.

Root cause

The `pyproject.toml` was written when the package root was the repository root, but the package was later moved into `backend/` without updating the readme path. No CI verifies that the documented install command actually works.

Suggested fix

Either (a) change `readme = "../README.md"` to `readme = "README.md"` and add a `README.md` inside `backend/`, or (b) change the documented install command to `PYTHONPATH=backend python -m maestro_api.main`. Add CI that runs the documented install + start command on a clean clone.

HIGH**#6 "Ask the Organization" is keyword substring search, not natural-language question answering****Steps to reproduce**

1) Start the server with the demo seed. 2) Call GET /api/oem/ask?q=Should+we+hire+more+engineers

Expected behavior

The system returns a synthesised answer that addresses the question, citing relevant laws and learning objects, with a confidence that reflects how well the question can be answered from the available evidence.

Actual behavior

The system returns: "Evidence from 1 learning object(s): CHURN: Hooli contract lost (confidence: 0.15)". The answer has no relationship to the question. The system matched the word "hire" (a stopword-trimmed token) against a learning object whose description contains "hire" (from a Slack thread about the Q3 hiring plan), and returned that learning object with a templated "Evidence from N learning object(s)" prefix.

Root cause

maestro_oem/decision.py answer_question() (line 251-347) splits the query on whitespace, filters tokens longer than 3 chars, and does a substring match against law statements, learning object descriptions, expert entity names, and bottleneck gate names. There is no LLM, no embedding, no semantic model, no synthesis. The method returns whatever matched, prefixed by a templated string.

Suggested fix

Either (a) integrate an LLM (OpenAI/Anthropic/local) that takes the matched evidence and synthesises an actual answer to the question, with the evidence as context; or (b) rename the feature to "Keyword Search" and stop marketing it as "Ask the Organization". Option (a) is what the product's marketing implies; option (b) is the honest minimum.

HIGH**#7 Decision Simulator is two linear formulas — every "input" except hire_count is silently ignored****Steps to reproduce**

1) Start the server. 2) Call POST /api/oem/simulate with body {"inputs": {"hire_count": 5, "budget": 1000000, "timeline": "6mo", "team": "platform"}}. 3) Vary budget, timeline, and team. Observe the predicted_p1_cluster_risk and predicted_decision_velocity_days outputs.

Expected behavior

Each input affects the prediction. Budget affects velocity; timeline affects risk decay; team selection scopes the simulation to that team's historical patterns.

Actual behavior

Only hire_count affects the output. $\text{predicted_p1} = \max(0.0, \text{base_p1} - \text{hire_count} * 0.02)$. $\text{predicted_velocity} = \max(0.5, \text{base_velocity} - \text{hire_count} * 0.1)$. All other inputs are echoed back in inputs_applied but have zero effect on the prediction. The code comment in simulation.py line 62 says "other inputs are accepted but do not change the prediction yet" — the "yet" has not been followed up.

Root cause

maestro_oem/simulation.py simulate() is 117 lines including docstrings and comments. The actual logic is two arithmetic expressions. There is no causal model, no team-scoped data, no budget-to-velocity mapping, no timeline-to-risk-decay function.

Suggested fix

Either (a) implement the additional input dimensions the UI exposes, or (b) remove the unused inputs from the UI and rename the feature "Hire-Count Sensitivity Estimator". A Fortune 100 CTO who runs a what-if with five inputs and observes that four of them do nothing will not trust the fifth.

HIGH**#8 "Money lost" estimates in the CEO briefing are fabricated — evidence_count * 2 with "h/week" appended****Steps to reproduce**

1) Start the server with the demo seed. 2) Call GET /api/oem/ceo-briefing. 3) Inspect the money.losses array.

Expected behavior

Each money-loss entry is derived from actual cost data: hourly rates, incident durations, opportunity-cost estimates, business-provided inputs. The estimate is explained and traceable to a business input.

Actual behavior

Each money-loss entry is a string of the form `f"~{lo.evidence_count * 2}h/week lost in approval delays"` (bottlenecks), `f"~{lo.evidence_count}h/week wasted on duplicate effort"` (duplicates), or `f"~{lo.evidence_count}h/week in incident response"` (incidents). The number is the count of signals that triggered the learning object, multiplied by 1 or 2, with "h/week" appended. There is no business input, no hourly rate, no actual time tracking.

Root cause

`maestro_api/routes/oem.py get_ceo_briefing()` (line 1280-onwards) builds the `money_losses` list by iterating learning objects and stamping each with a templated cost string. The multiplication by 2 for bottlenecks is unexplained. The "h/week" unit is unexplained. No business stakeholder was asked whether 1 signal = 1 hour per week is a reasonable conversion.

Suggested fix

Either (a) remove the `estimated_cost` field from the CEO briefing and replace it with a qualitative severity tag (high/medium/low) until real cost data is available, or (b) integrate with a time-tracking system (Toggl, Harvest, Jira time logs) and derive cost from actual hours. Until then, presenting fabricated numbers to a CEO is a trust-destroying defect.

HIGH**#9 Organisational laws are promoted from pattern to "law" after 3 observations — three is not a law****Steps to reproduce**

1) Start the server with the demo seed. 2) Call GET /api/oem/laws. 3) Observe the laws and their `validated_runtimes` counts.

Expected behavior

Laws are promoted only after a statistically meaningful number of observations across multiple providers, teams, and time windows. The promotion threshold reflects the variance of the underlying signal.

Actual behavior

Laws are promoted after 3 validated runtimes (`maestro_oem/law.py OrganizationalLaw.add_validation()`: `"if self.status == LawStatus.CANDIDATE and self.validated_runtimes >= 3: self.status = LawStatus.VALIDATED"`). The demo dataset produces laws with confidence 1.0 from three Slack messages. A CEO shown a "law" with confidence 1.0 based on three observations is being misled.

Root cause

The threshold of 3 is a magic number with no statistical justification. The confidence formula (Bayesian with uniform prior) returns 1.0 for 3-of-3 because $\alpha=1+3$, $\beta=1+0$, $\text{posterior_mean} = 4/4 = 1.0$. A uniform prior is appropriate for "no prior knowledge" but inappropriate for "promote to law".

Suggested fix

Raise the promotion threshold to at least 20 observations across at least 2 providers and 2 teams. Use a `Beta(0.5, 0.5)` (Jeffreys) prior instead of `Beta(1, 1)` for law promotion, so 3-of-3 produces ~ 0.85 confidence, not 1.0. Add a `"law_candidate"` status that surfaces in the UI as a hypothesis, not a law.

HIGH**#10 Supabase and Auth0 OAuth providers are stubs that return None on every method****Steps to reproduce**

1) Set MAESTRO_OAUTH_PROVIDER=supabase (or auth0) with valid client_id and client_secret. 2) Attempt a login.

Expected behavior

The user is authenticated via Supabase or Auth0.

Actual behavior

SupabaseProvider.verify_token() logs "Supabase OAuth not yet implemented in v1.0; use API key auth" and returns None. SupabaseProvider.exchange_code() logs "Supabase OAuth code exchange not yet implemented in v1.0" and returns None. Auth0Provider has the same stubs. The user cannot log in.

Root cause

maestro_auth/oauth.py SupabaseProvider and Auth0Provider are explicitly documented as stubs for v1.0, with TODO(v1.1) comments. The README lists Supabase and Auth0 as supported authentication methods without disclosing that they do not work.

Suggested fix

Either (a) implement Supabase and Auth0 OAuth fully, or (b) remove them from the README and docs and the make_provider() factory until they are implemented. The current state — listing non-functional providers in the documentation — is a procurement-failing defect.

HIGH**#11 EncryptionManager silently falls back to XOR "encryption" if cryptography is not installed****Steps to reproduce**

1) Deploy without the cryptography package installed (it is in pyproject.toml but not enforced at runtime). 2) Store an OAuth client secret via the OAuth admin API. 3) Read the stored secret back.

Expected behavior

The EncryptionManager raises a RuntimeError at startup if cryptography is not available. No secret is ever stored with XOR obfuscation.

Actual behavior

EncryptionManager._get_fernet() catches ImportError, logs "cryptography not installed — using insecure XOR fallback", and returns None. EncryptionManager.encrypt() then falls through to `return "xor:" + base64(...)` where the body is `bytes(a ^ b for a, b in zip(plaintext.encode(), self.\_key \* 100))`. This is reversible by anyone who has the source code (which is open-source on GitHub).

Root cause

maestro_auth/security.py EncryptionManager was designed to "fail-closed in production" (line 460: "NEVER auto-generate or fall back to a plaintext key in production") but the same class falls back to XOR obfuscation for the encrypt/decrypt methods themselves. The fail-closed mandate applies to key generation, not to cipher selection.

Suggested fix

Make the cryptography package a hard dependency (it is already declared; add an explicit `import cryptography` at module load that raises if missing). Remove the XOR fallback entirely. If cryptography is not installed, EncryptionManager.encrypt() should raise RuntimeError, not silently degrade.

HIGH #12 OAuth callback returns raw JSON instead of redirecting to a UI page

Steps to reproduce

1) Configure a real OAuth provider (e.g. GitHub). 2) Click Connect in the Settings page. 3) Complete the OAuth round-trip on the provider's site. 4) Observe the page the user lands on after the callback.

Expected behavior

The user is redirected to the Settings page (or a connection-success page) with a visible "GitHub connected" indicator.

Actual behavior

The user lands on a raw JSON response: `{"ok": true, "provider": "github", "connected": true, ...}`. There is no HTML, no redirect, no UI feedback. The frontend has no code that consumes this response (the OAuth flow uses `window.location.href`, not `fetch`, so there is no JS handler waiting for the callback). The user is left looking at JSON in their browser with no indication that anything happened.

Root cause

`maestro_api/routes/imports.py` `oauth_callback()` returns a plain dict, which FastAPI serialises as JSON. There is no `RedirectResponse`, no HTML template, no frontend route that consumes the callback result.

Suggested fix

Change `oauth_callback()` to return `RedirectResponse(url="/#/eng-settings?connected=github")` (or similar). Add a frontend handler that reads the `connected` query param and shows a success toast.

HIGH #13 In-process weekly snapshot scheduler runs per-replica in multi-replica deployments

Steps to reproduce

1) Deploy MaestroAgent with multiple replicas (e.g. 3 uvicorn workers behind a load balancer). 2) Wait 7 days. 3) Inspect the `SnapshotStore` for snapshot records.

Expected behavior

One snapshot is recorded per week, regardless of replica count.

Actual behavior

Three snapshots are recorded per week (one per replica). Each replica's `lifespan()` spawns its own `_weekly_snapshot_loop()` `asyncio` task with `asyncio.sleep(7 * 24 * 60 * 60)`. There is no distributed lock, no leader election, no singleton enforcement.

Root cause

`maestro_api/main.py` `lifespan` (line 96-122) spawns the snapshot loop with the comment "In production, use a real cron scheduler; this in-process loop is sufficient for single-instance pilots." The comment acknowledges the limitation but the code is shipped as the production default.

Suggested fix

Either (a) use a distributed lock (e.g. PostgreSQL advisory lock, Redis `SETNX`) so only one replica runs the snapshot per week, or (b) move the snapshot to an external cron job (the code already supports this — `collect_snapshot_metrics()` can be called from a CLI) and remove the in-process loop. Update the deployment docs accordingly.

MEDIUM (workflow problems)



#14 Demo seed data is not removed when a real provider is connected

Steps to reproduce

1) Start the server with MAESTRO_DEMO_SEED=true (default). 2) Connect a real GitHub provider. 3) Wait for historical ingestion to complete. 4) Call GET /api/oem/recommendations.

Expected behavior

Recommendations are derived from the real GitHub data only. The acme-corp synthetic data is removed or quarantined.

Actual behavior

Recommendations are derived from the union of the acme-corp synthetic data and the real GitHub data. priya.m@acme.com still appears as a hidden expert. The "P1 cluster risk: 45% probability of velocity drop" recommendation still appears, derived from the synthetic EMEA Jira tickets. The user has no way to remove the synthetic data without wiping the OEM database.

Root cause

maestro_api/oem_state.py live_ingest() appends new signals to self.signals and ingests them into the existing model. There is no code path that removes the demo seed when a real connection is established. The README says "When real providers are connected via OAuth, their live signals are ingested on top via live_ingest() — the demo seed is NOT removed automatically" — the limitation is documented but not solved.

Suggested fix

On the first successful OAuth connection, automatically wipe the demo seed (set oem_state.signals = [], rebuild the model from scratch, re-ingest real signals). Add a UI confirmation: "You have connected a real provider. Remove the demo data?" with Yes/No. If No, mark the demo signals with a demo=true flag and exclude them from recommendations.



#15 Drill-down modal loops for several entity types — clicking a card inside the modal reopens the same modal

Steps to reproduce

1) Open the Home page. 2) Click the "Concentration risk in deployment" card in the Hayek Lens section. 3) In the drill-down modal, switch to the "Where?" tab. 4) Click the concentration-risk card shown in that tab.

Expected behavior

The "Where?" tab shows the underlying signals, people, and timeline that produced the concentration risk. Clicking any sub-card opens a deeper drill-down (e.g. the person's expert profile) or closes the modal.

Actual behavior

The "Where?" tab re-renders the same concentration-risk card the user just clicked. Clicking it calls openDrilldown('risk', 'deployment') again, which reopens the same modal with the same tabs. The user is in a click loop with no terminal view.

Root cause

drill_down_modal.js switchDrilldownTab() does not differentiate the "Where?" tab content by entity type. For some entity types (recommendations, laws), "Where?" shows useful context. For others (risks, experts), "Where?" falls back to the same card that was clicked.

Suggested fix

For each entity type, define what "Where?" should show: for a risk, the list of people who hold the concentrated knowledge; for an expert, the list of domains they touch; for a bottleneck, the list of items currently gated. Render that list, not the parent card. If no "Where?" data exists for an entity type, hide the tab.

MEDIA M #16 "Live Meeting" requires manual transcript paste — there is no live audio or meeting platform integration

Steps to reproduce

1) Open the Live Meeting page. 2) Click "Start Live Meeting".

Expected behavior

The page connects to an active meeting (Zoom, Teams, Meet), captures audio, transcribes in real time, and surfaces objections / laws triggered / action items as the meeting progresses.

Actual behavior

The page displays: "Paste a transcript below and click 'Analyze with OEM' — the OEM will detect objections, laws triggered, and action items in real time." A textarea appears. The user must manually type or paste a transcript. There is no audio capture, no meeting platform integration, no real-time transcription. The "live" branding is a misnomer.

Root cause

static/js/live_meeting.js startLiveMeeting() shows a textarea and a button. The analyzeTranscript() function parses the pasted text line-by-line ("Speaker: text") and calls /api/oem/meetings/analyze. The "real time" claim refers to the analysis being synchronous after paste, not to live capture.

Suggested fix

Either (a) integrate a transcription service (Deepgram, Whisper, AssemblyAI) and a meeting platform (Zoom OAuth, Teams Graph API, Meet Capture API), or (b) rename the feature to "Transcript Analyzer" and remove the "live" branding. The current name is misleading.

MEDIA M #17 COVERAGE_REPORT.md claims "100k engineers (scale) □ test_large_history_simulation (2000 events)" — claim and test disagree by 50x

Steps to reproduce

1) Open docs/COVERAGE_REPORT.md. 2) Find the "Scenario Coverage" table. 3) Open backend/maestro_oem/tests/test_historical_engine.py. 4) Find test_large_history_simulation().

Expected behavior

The COVERAGE_REPORT claim matches the test: either the test simulates 100k events, or the report claims 2000 events.

Actual behavior

The COVERAGE_REPORT claims "100k engineers (scale) test_large_history_simulation (2000 events)". The test docstring says "Simulate 100k events (1000 pages of 100)". The test body creates _TestFetcher(oauth, total_pages=20, page_size_items=100) and asserts len(received) == 2000. The test simulates 2000 events, not 100k.

Root cause

The test was written with a placeholder page count (20 pages) and never updated to the documented 1000 pages. The COVERAGE_REPORT was generated from the test docstring, not from the test body.

Suggested fix

Either (a) update the test to actually simulate 100k events (1000 pages of 100) and verify the engine handles it within a reasonable time/memory budget, or (b) update the COVERAGE_REPORT to claim "2000 events" honestly. Do not claim 100k if the test only runs 2k.

MAESTRO #18 CORS allows credentials with wildcard origins when auth is disabled (default dev mode)

Steps to reproduce

1) Start the server in development mode (default, MAESTRO_AUTH_ENABLED unset). 2) From any origin, make a credentialed fetch to /api/oem/dashboard.

Expected behavior

CORS rejects credentialed requests from arbitrary origins.

Actual behavior

maestro_api/main.py create_app() line 144-151: when auth_config.cors_origins is empty (the default in dev mode), the middleware is configured with allow_origins=["*"], allow_credentials=False (because cors_origins != ["*"] is False). This is correct. However, the same code path sets allow_methods=["*"] and allow_headers=["*"] which is overly permissive. More importantly, if MAESTRO_CORS_ORIGINS is set to a single origin in production but MAESTRO_AUTH_ENABLED is unset, allow_credentials=True is set but the origin list is a single entry — this is fine. The defect is that the default dev-mode CORS is wildcard, which is correct for dev but should be documented as a production anti-pattern.

Root cause

The CORS configuration is environment-aware but the awareness is implicit. There is no startup warning that says "CORS is wildcard — do not deploy this configuration to production".

Suggested fix

Add a startup warning when allow_origins == ["*"] and MAESTRO_ENV != "development". Refuse to start in production with wildcard CORS.

LOW (polish)

LOW #19 self-graded "9.7/10" Fortune 100 Readiness Report is contradicted by the repository's own root README

Steps to reproduce

1) Open docs/FORTUNE_100_READINESS_REPORT.md. 2) Read the disclaimer at the top. 3) Open the root README.md.

Expected behavior

The readiness report either does not exist (because it was retracted) or it has been independently verified by a third party.

Actual behavior

The report still exists with all original scores (Engineering 9.7/10, Security 9.8/10, Overall 9.7/10) but carries a disclaimer at the top admitting it was self-graded by the build process and that an external audit found a committed encryption master key that the 9.8/10 security score did not catch.

Root cause

The report was generated by a build script that ran tests and assigned scores. It was never independently verified. When the external audit found the committed key, a disclaimer was prepended rather than the report being retracted.

Suggested fix

Delete docs/FORTUNE_100_READINESS_REPORT.md and docs/QA_REPORT_FINAL.md. They are misleading even with disclaimers. Replace with a single docs/KNOWN_ISSUES.md that lists the actual open issues.

LOW**#20 app.html size and CSS size exceed the limits asserted by the project's own test_comprehensive_qa.py****Steps to reproduce**

1) Run `python -m pytest`

`backend/maestro_api/tests/test_comprehensive_qa.py::TestPerformanceMetrics::test_app_html_under_60kb`

`backend/maestro_api/tests/test_comprehensive_qa.py::TestPerformanceMetrics::test_css_under_25kb --tb=short`

Expected behavior

Both tests pass.

Actual behavior

Both tests fail. `test_app_html_under_60kb` asserts the HTML file is under 60 KB; the actual `app.html` is ~64 KB.

`test_css_under_25kb` asserts the CSS is under 25 KB; the actual `app.css` is ~32 KB. The limits were set when the files were smaller; the files grew and the limits were not updated.

Root cause

The performance budgets were set aspirationally and not enforced by CI.

Suggested fix

Either (a) trim `app.html` and `app.css` to fit the budgets (remove inline styles, consolidate CSS), or (b) raise the budgets to 70 KB and 35 KB and document why. Budgets that are not enforced are worse than no budgets.

LOW**#21 Deprecated frontend code is shipped in the repository and inflates the clone size by 4.5 MB****Steps to reproduce**

1) git clone the repository. 2) `du -sh download/MaestroAgent/_deprecated/`

Expected behavior

The `_deprecated/` directory is not in the repository.

Actual behavior

The `_deprecated/` directory is 4.5 MB and contains abandoned Tauri+React, Vite+React, Next.js, and Next.js+Prisma+Redis frontends plus a Node.js realtime-server. The README says "Do not run anything in `_deprecated/`" but the code is still shipped.

Root cause

The deprecated code was kept "for reference" but serves no purpose in a release. It increases clone time, confuses search, and presents a maintenance risk if anyone copies from it.

Suggested fix

Move `_deprecated/` to a separate archive repository or remove it from the main repository entirely. The git history preserves it for reference.

LOW**#22 README install command references `python -m maestro_api.main` but the package is named `maestroagent` and the entry point is the `maestro CLI`****Steps to reproduce**

1) Read download/MaestroAgent/README.md. 2) Read backend/pyproject.toml [project.scripts].

Expected behavior

The README install command and the pyproject.toml entry point agree.

Actual behavior

README says: ``python -m maestro_api.main``. pyproject.toml [project.scripts] declares: ``maestro = "maestro_cli.main:app"``. The two are inconsistent — one runs the API server, the other runs the CLI. The README does not mention the `maestro CLI`.

Root cause

The README was written for the API-only deployment and never updated to mention the CLI entry point.

Suggested fix

Document both: ``python -m maestro_api.main`` for the API server, ``maestro`` for the CLI. Add a "Quick Start" section that covers both.

LOW**#23 No CI/CD pipeline configured — tests are run manually and the "837+ tests pass" claim cannot be verified by contributors****Steps to reproduce**

1) Check the repository root for `.github/workflows/`, `.gitlab-ci.yml`, `.circleci/config.yml`, or `Jenkinsfile`. 2) Check for any automated test execution on push or pull request.

Expected behavior

A CI pipeline runs the test suite on every push and pull request, and the result is visible in the repository.

Actual behavior

No CI configuration exists in the repository. The "837+ tests pass on a clean clone" claim is unverifiable by anyone who does not manually run the suite. The 18 test failures observed in this audit would have been caught by CI on the first push after they were introduced.

Root cause

No CI was ever set up. The `COVERAGE_REPORT.md` acknowledges this under "Deductions (-0.3): No CI/CD pipeline configured (tests run manually)".

Suggested fix

Add a GitHub Actions workflow that runs `python -m pytest` on every push and pull request. Run it on Python 3.11 and 3.12. Block merges on test failures.

Final Scores

Scores are out of 10. They reflect what was observed in this audit, not what the product's own documentation claims. Where the user's prompt asked for browser-only measurements (First Paint, LCP, animation smoothness, CPU usage under 30 tabs), the audit substitutes the closest source-level evidence and notes the substitution.

Category	Score	Justification (one-line)
Navigation	7/10	All 14 sidebar items render and navigate. Hash routing works. Breadcrumbs update. No dead links. Loses 3 p
Usability	5/10	Click every card and a drill-down opens. But the drill-down loops, the OAuth callback lands on raw JSON (I
Enterprise Readiness	2/10	Multi-tenant isolation not implemented (Issue #2). OIDC signature verification skipped (Issue #1). Supabase
Interaction Quality	6/10	Autocomplete is mechanically solid (AbortController, keyboard nav, SWR cache). But the underlying engine
Performance	6/10	Frontend is genuinely lightweight (64 KB HTML, 16.5 KB CSS, no framework, no CDN). Backend has O(n
Reliability	4/10	18 tests fail on a clean clone (Issue #3). pip install -e backend/ fails (Issue #5). The README's "837+ tests p
Accessibility	4/10	Skip-link, role="dialog", tabindex, focus management — all present. But four ARIA tests fail on a clean clon
Data Credibility	3/10	Concentration risks and hidden experts are believably inferable. But organisational laws are promoted from 3
Execution Flow	5/10	OAuth round-trip is real. Historical ingestion is real. Closed-loop learning is real (prediction_lifecycle.py). B
Overall Production Readiness	3/10	The architecture is sound. The intent is honest. The execution is incomplete. The security defects (Issues #1,

Final Verdict

The user's prompt requires answering exactly one question: "Would you ship this to a Fortune 100 customer tomorrow?" The allowed answers are YES, YES WITH MINOR FIXES, NO, or ABSOLUTELY NOT.

ABSOLUTELY NOT

Rationale. The four CRITICAL issues are each independently disqualifying. OIDC JWT signature verification is silently skipped (Issue #1) — any deployed instance without PyJWT installed accepts forged tokens. Multi-tenant isolation is not implemented at the data layer (Issue #2) — every user sees every other user's data in a multi-tenant deployment. The README's "837+ tests pass on a clean clone" claim is false (Issue #3) — 18 tests fail in three sampled files. The inline onclick handlers pass server-controlled strings into JS string literals using an escaping function that is incorrect for that context (Issue #4) — any signal title containing an apostrophe is a code-injection vector.

The HIGH issues compound the disqualification. "Ask the Organization" is keyword search marketed as NL question answering (Issue #6). The Decision Simulator is two linear formulas marketed as a what-if engine (Issue #7). Money-loss estimates are fabricated (Issue #8). Organisational laws are promoted from three observations (Issue #9). Supabase and Auth0 OAuth are stubs (Issue #10). The encryption manager silently degrades to XOR (Issue #11). The OAuth callback lands on raw JSON (Issue #12). The snapshot scheduler duplicates across replicas (Issue #13). pip install -e backend/ fails on a clean clone (Issue #5).

What would be required to change the verdict. Fix Issues #1, #2, #3, #4, #5, #11 (release blockers) and re-run the test suite on a clean clone with 0 failures. Fix Issues #6, #7, #8, #9 (procurement blockers) or downgrade the marketing to match the implementation. Implement CI (Issue #23). At that point, the verdict would move to NO. To move to YES WITH MINOR FIXES, additionally fix Issues #10, #12, #13, #14, #15 and have an independent third party re-audit. To move to YES, run the system against a real Fortune 100 deployment for 90 days and publish the results.

What this audit did not find. The audit did not find a fundamentally broken architecture. The signal-to-learning-object-to-pattern-to-law-to-recommendation pipeline is well-designed. The closed-loop learning (prediction lifecycle, calibration, Brier score) is genuinely implemented. The evidence graph is real. The confidence formula is documented and applied. The security middleware stack (CSRF, CSP, rate limiting, tenant isolation, audit chain) exists on paper. The authors' self-critique document (REASONS_WE_MIGHT_BE_WRONG.md) is unusually honest. The defect is not that the product is vapourware; the defect is that the product is a v0.3 dressed in v1.0 clothing. The gap between the marketing and the implementation is the disqualifier, not the implementation itself.

A note to the MaestroAgent authors. This audit is brutal because the user's prompt demanded brutality and because the stakes (a \$2M/year Fortune 100 contract) require it. The codebase shows real engineering judgement and real intellectual honesty in places (the self-critique document, the disclaimers on the self-graded reports, the comments acknowledging known limitations). The path from ABSOLUTELY NOT to YES is long but visible: fix the security release blockers, narrow the marketing to match the implementation, run CI on every push, and re-audit in 90 days. Ship v0.4 honestly before shipping v1.0 dishonestly.